

IA PARA DEVS

PROCESSAMENTO DE LINGUAGEM NATURAL

AULA 02

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	10
O QUE VOCÊ VIU NESTA AULA?	16
REFERÊNCIAS.....	17

EMANIP

O QUE VEM POR AÍ?

Bem-vindos e bem-vindas à nossa aula sobre Bag of Words e Word Cloud! Aqui, implementaremos um classificador de sentimento utilizando o método de regressão logística para prever frases, porém vamos encontrar um problema, o modelo de Machine Learning não entende texto, e com isso exploraremos o conceito de Bag of Words para conseguirmos criar um modelo funcional.



HANDS ON

Nesta aula aprendemos a como implementar um classificador de sentimentos utilizando o Bag of Words. A seguir veremos o código de cada célula que fizemos no Google Colab. Não deixe de praticar e revisar o conteúdo!

Primeiro nós baixamos o dataset no [kaggle](#).

```
import pandas as pd

avaliacoes = pd.read_csv("b2w.csv")

avaliacoes.head()
```

Visualizamos os dados.

```
avaliacoes = avaliacoes.drop(["original_index", "review_text_processed",
"review_text_tokenized", "rating", "kfold_polarity", "kfold_rating"], axis=1)
```

Limpamos as colunas desnecessárias.

```
avaliacoes.head()
```

Visualizamos os 5 primeiros registros.

```
avaliacoes.dropna(inplace=True, axis=0)

avaliacoes
```

Retiramos os registros com dados NaN (Not a Number) e visualizamos novamente.

```
avaliacoes.polarity.value_counts()
```

Observamos a quantidade de dados de cada tipo de polaridade.

```
from sklearn.model_selection import train_test_split

treino, teste, classe_treino, classe_teste = train_test_split(avaliacoes.review_text,
                                                             avaliacoes.polarity,
                                                             stratify=avaliacoes.polarity,
                                                             random_state = 71)
```

Separamos os dados em treino e teste (lembrando do parâmetro stratify que observa a desigualdade entre os dados).

```
from sklearn.linear_model import LogisticRegression

# regressao_logistica = LogisticRegression()
# regressao_logistica.fit(treino, classe_treino)
# acuracia = regressao_logistica.score(teste, classe_teste)
# print(acuracia)
```

Nessa célula fizemos a tentativa de treinar o modelo, mas sem sucesso, pois o computador não entende os dados da maneira que estão, por isso está comentado e só está importando a biblioteca LogisticRegression para usarmos posteriormente.

```
from sklearn.feature_extraction.text import CountVectorizer

texto = ["Este produto é muito bom", "Este produto é muito ruim"]

vetorizar = CountVectorizer()

bag_of_words = vetorizar.fit_transform(texto)
```

Começamos a entender o como funciona o bag of words com este código.

```
matriz_esparsa = pd.DataFrame.sparse.from_spmatrix(bag_of_words,
                                                    columns=vetorizar.get_feature_names_out())
```

Aqui nós definimos nossa matriz esparsa.

```
vetorizar = CountVectorizer(max_features=100)
bag_of_words = vetorizar.fit_transform(avaliacoes.review_text)
print(bag_of_words.shape)
```

Vetorizamos e definimos a quantidade de colunas com dados mais recorrentes que foram usados no modelo.

```
def treinar_modelo(dados, coluna_texto, coluna_sentimento):
    vetorizar = CountVectorizer(max_features=100)
    bag_of_words = vetorizar.fit_transform(dados[coluna_texto])
    treino, teste, classe_treino, classe_teste = train_test_split(bag_of_words,
                                                                    dados[coluna_sentimento],
                                                                    stratify=dados[coluna_sentimento],
                                                                    random_state = 71)
    regressao_logistica = LogisticRegression()
    regressao_logistica.fit(treino, classe_treino)
    return regressao_logistica.score(teste, classe_teste)
```

Criamos um método para criar e testar o modelo, lembrando que o parâmetro `random_state` é para garantir a reprodutibilidade dos treinos e obtermos os mesmos resultados.

```
print(treinar_modelo(avaliacoes, "review_text", "polarity"))
```

Printamos a acurácia do modelo

```
from wordcloud import WordCloud

todas_avaliacoes = [texto for texto in avaliacoes.review_text]

todas_palavras = ' '.join(todas_avaliacoes)
```

Aqui nós importamos o Word Cloud e separamos as frases em um array e depois juntamos tudo em uma string separada por espaço, criando uma string com todas as palavras do nosso corpus textual.

```
nuvem_palavras = WordCloud(width=800, height=500,
                             max_font_size=110,
                             collocations=False).generate(todas_palavras)
```

Criamos a nossa word cloud com a variável `nuvem_palavras` e definimos o parâmetro `collocations` como `false` para evitar Ngrams que são conjuntos de palavras, como por exemplo: “o produto”, “não recebi”, “muito bom”, “muito ruim”, etc.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10,7))
plt.imshow(nuvem_palavras, interpolation='bilinear')
plt.axis("off")
plt.show()
```

Plotamos nosso gráfico da word cloud com todas as palavras juntas.

```
avaliacoes.query("polarity == 1")
```

Entendemos como filtrar dados dentro de um dataframe da biblioteca pandas.

```
def word_cloud_neg(dados, coluna_texto):  
    texto_negativo = dados.query("polarity == 0")  
    todas_avaliacoes = [texto for texto in texto_negativo[coluna_texto]]  
    todas_palavras = ' '.join(todas_avaliacoes)  
  
    plt.figure(figsize=(10,7))  
    plt.imshow(nuvem_palavras, interpolation='bilinear')  
    plt.axis("off")  
    plt.show()
```

Criamos um método para criar a word cloud com palavras das sentenças negativas do nosso dataset.

```
def word_cloud_pos(dados, coluna_texto):  
    texto_positivo = dados.query("polarity == 1")  
    todas_avaliacoes = [texto for texto in texto_positivo[coluna_texto]]  
    todas_palavras = ' '.join(todas_avaliacoes)  
  
    plt.figure(figsize=(10,7))  
    plt.imshow(nuvem_palavras, interpolation='bilinear')  
    plt.axis("off")  
    plt.show()
```

E criamos para a word cloud de palavras das sentenças positivas do nosso dataset.

```
word_cloud_neg(avaliacoes, "review_text")
```

Por fim, plotamos o gráfico com as palavras de sentenças positivas.

```
word_cloud_pos(avaliacoes, "review_text")
```

E das sentenças negativas do dataset.

Explore mais o código e leia a documentação para obter mais informações sobre os métodos e recursos.



SAIBA MAIS

O Bag of Words (BoW) é uma técnica fundamental em Processamento de Linguagem Natural (PLN) que representa documentos de texto como conjuntos não ordenados de palavras, ignorando a estrutura gramatical e a ordem das palavras.

Aqui estão alguns pontos-chave para uma introdução ao Bag of Words:

Representação Simples

O BoW trata cada documento como uma "sacola" (bag) de palavras, desconsiderando completamente a ordem em que as palavras aparecem.

A representação é simplificada, transformando o texto em vetores que indicam a presença ou contagem de palavras específicas.

Processo de Construção

- **Tokenização:** inicia-se com a divisão do texto em unidades individuais, conhecidas como tokens. Geralmente, os tokens são palavras, mas podem ser n-gramas (sequências de n palavras).
- **Construção do Vocabulário:** cria-se um vocabulário único contendo todas as palavras distintas nos documentos. Cada palavra recebe um identificador único.
- **Vetorização:** cada documento é, então, representado como um vetor, onde cada dimensão corresponde a uma palavra do vocabulário. A entrada do vetor pode ser a contagem de ocorrências da palavra no documento ou uma variável binária indicando a presença ou ausência da palavra.
- **Esparsidade:** os vetores resultantes são frequentemente esparsos, já que a maioria das palavras em um vocabulário pode não estar presente em um documento específico.
- **Utilização em Modelos de ML:** os vetores BoW são frequentemente utilizados como entrada para modelos de aprendizado de máquina em tarefas como classificação de texto, análise de sentimento e recuperação de informações.

- **Limitações:** a principal limitação é a perda de informações de sequência e contexto. A estrutura gramatical e a ordem das palavras não são consideradas. Palavras frequentes podem dominar a representação, e informações semânticas mais profundas podem ser perdidas.
- **Eficiência Computacional:** o BoW é computacionalmente eficiente e fácil de implementar. Isso o torna útil em cenários onde a simplicidade é preferida ou onde o conjunto de dados é grande.
- **Pré-processamento Importante:** o sucesso do BoW muitas vezes depende do pré-processamento adequado, incluindo remoção de stop words, stemming ou lematização e tratamento de caracteres especiais.
- **Exemplo Simples:** para ilustrar, considere os documentos " Este produto é muito bom" e " Este produto é muito ruim". O vocabulário seria ["Este", "produto", "é", "muito", "bom", "ruim"], e os documentos seriam representados como vetores indicando a contagem de cada palavra.

	Este	produto	é	muito	bom	ruim
Este produto é muito bom	1	1	1	1	1	0
Este produto é muito ruim	1	1	1	1	0	1
Este produto é bom bom bom	1	1	1	0	3	0

Figura 1 – Exemplo simples
Fonte: Elaborado pelo autor (2024)

Aplicações Além do Texto

Além de PLN, o conceito de BoW é usado em outras áreas, como processamento de imagem (usando histogramas de cores) e em sistemas de recomendação.

O Bag of Words, embora simples, é uma ferramenta poderosa para muitas tarefas de PLN, fornecendo uma representação numérica de documentos de texto que pode ser facilmente utilizada em algoritmos de aprendizado de máquina.

A construção do vocabulário é uma etapa crucial na abordagem do Bag of Words (BoW). Esta fase envolve a criação de um conjunto único de palavras presentes nos documentos, que será usado para representar os documentos como vetores. Aqui

estão alguns pontos importantes sobre a construção do vocabulário no contexto do BoW:

- **Tokenização:** o processo começa com a tokenização, que envolve a divisão do texto em unidades menores, chamadas de tokens. Em geral, os tokens são palavras individuais, mas podem incluir também n-gramas (sequências de n palavras consecutivas).
- **Remoção de Stop Words:** muitas vezes, as palavras mais comuns em um idioma (chamadas de stop words), como "o", "a", "e", são removidas do vocabulário. Isso ajuda a reduzir a dimensionalidade e focar em palavras mais informativas.
- **Minimização de Redundâncias:** palavras podem ser lematizadas ou reduzidas ao seu tronco (stemming) para minimizar redundâncias. Por exemplo, "correr" e "correndo" podem ser reduzidos a "corr".
- **Contagem ou Binarização:** ao construir o vocabulário, decide-se se a representação dos documentos será baseada na contagem de ocorrências de palavras, ou se será uma representação binária indicando apenas a presença ou ausência de cada palavra.
- **Ordem Alfabética ou Outra Ordem:** o vocabulário pode ser organizado alfabeticamente ou por frequência de palavras, dependendo da preferência ou da necessidade de otimização.
- **Identificadores Únicos:** cada palavra no vocabulário é associada a um identificador único, geralmente um índice inteiro. Esses identificadores são usados para criar os vetores de documentos.
- **Tamanho do Vocabulário:** o tamanho do vocabulário pode variar dependendo do conjunto de dados e dos requisitos da aplicação. Em grandes conjuntos de dados, o vocabulário pode ser extenso, levando a vetores de alta dimensionalidade.
- **Processamento Eficiente:** em casos práticos, é importante realizar esse processo de construção do vocabulário de maneira eficiente, especialmente em grandes volumes de dados.

- **Exclusão de Palavras Raras:** em alguns casos, palavras muito raras podem ser excluídas do vocabulário para evitar a superespecificação do modelo e melhorar a generalização.
- **Atualização Dinâmica:** em ambientes dinâmicos, o vocabulário pode ser atualizado à medida que novos documentos são introduzidos para garantir que o modelo leve em consideração novos termos e conceitos.

A construção do vocabulário é uma etapa fundamental na criação de modelos BoW eficazes. Um vocabulário bem construído é essencial para garantir que os documentos sejam representados de maneira significativa e que a análise subsequente seja informativa.

Uma Word Cloud (nuvem de palavras) é uma representação visual de palavras em um conjunto de dados de texto, onde o tamanho de cada palavra é proporcional à sua frequência no texto. Essa técnica fornece uma visão rápida e intuitiva das palavras mais significativas ou frequentes em um determinado contexto. Aqui estão alguns pontos importantes sobre o conceito de Word Cloud:

- **Representação Gráfica:** a Word Cloud é uma representação gráfica das palavras mais frequentes em um corpus de texto, onde as palavras são exibidas em diferentes tamanhos e cores.
- **Frequência das Palavras:** o tamanho de cada palavra na nuvem é determinado pela sua frequência no texto. Palavras mais frequentes aparecem em um tamanho maior, enquanto palavras menos frequentes são exibidas em um tamanho menor.
- **Visualização Intuitiva:** a Word Cloud oferece uma maneira intuitiva de identificar padrões e tendências no texto, destacando visualmente as palavras mais proeminentes.
- **Ignorando Stop Words:** stop words (palavras de parada), como "o", "a", "e", geralmente são removidas antes de criar uma Word Cloud para destacar palavras mais informativas.
- **Personalização Estética:** a estética da Word Cloud pode ser personalizada, permitindo a escolha de cores, formas e layouts para tornar a visualização mais atraente e informativa.

- **Facilidade de Interpretação:** pode ser usada como uma ferramenta de comunicação eficaz, pois permite que as pessoas vejam rapidamente as palavras-chave em um conjunto de dados de texto sem ter que analisar o texto completo.
- **Aplicações em Análise de Texto:** as Word Clouds são frequentemente usadas em análises exploratórias de dados, resumos visuais de tópicos de discussão, análise de sentimentos e na identificação de temas dominantes em grandes volumes de texto.
- **Ferramentas e Bibliotecas:** existem várias ferramentas online e bibliotecas de programação que facilitam a criação de Word Clouds. Alguns exemplos incluem Wordle, TagCrowd, e bibliotecas como wordcloud em Python.
- **Palavras-Chave em Documentos:** além de análises gerais de conjuntos de dados, as Word Clouds também podem ser úteis para destacar palavras-chave em documentos específicos, oferecendo uma visão rápida do conteúdo principal.
- **Limitações:** embora as Word Clouds sejam úteis para fornecer uma visão geral, elas têm limitações, como a falta de informações sobre contextos mais complexos e relacionamentos entre palavras.

As Word Clouds são uma ferramenta acessível e eficaz para visualizar rapidamente as palavras mais importantes em um conjunto de dados de texto, tornando-as populares em uma variedade de contextos, desde análise de dados até apresentações visuais de informações textuais.

A construção de uma Word Cloud envolve vários passos, desde a preparação do texto até a geração da visualização final. Aqui estão os principais aspectos envolvidos na construção de Word Clouds:

- **Ponderação:** algumas implementações de Word Clouds oferecem opções para ponderar as palavras de acordo com critérios específicos, como relevância ou importância contextual.
- **Escolha de Layout e Formato:** o usuário pode escolher o layout e o formato da Word Cloud, como a forma da nuvem e a disposição das

palavras. Existem ferramentas e bibliotecas que fornecem opções para personalização estética.

- **Atribuição de Cores:** as cores podem ser atribuídas às palavras com base em critérios, como intensidade da cor relacionada à frequência ou categorização de palavras por temas.
- **Geração da Word Cloud:** utilizando uma ferramenta específica ou uma biblioteca de programação, a Word Cloud é gerada com base nas frequências das palavras. As palavras mais frequentes aparecerão em tamanho maior.
- **Exclusões Personalizadas:** algumas implementações permitem que o usuário exclua palavras específicas ou aplique personalizações adicionais, como a inclusão de palavras compostas ou termos específicos.
- **Interpretação e Análise:** após a geração da Word Cloud, a visualização resultante pode ser interpretada para extrair insights sobre os temas ou tópicos mais importantes no texto. Palavras maiores e mais destacadas são aquelas que ocorrem com mais frequência.
- **Atualizações Dinâmicas:** em casos dinâmicos, onde o conjunto de dados é alterado ao longo do tempo, é possível atualizar a Word Cloud para refletir novas informações.
- **Exportação e Compartilhamento:** a Word Cloud finalizada pode ser exportada como uma imagem ou incorporada em apresentações, relatórios ou sites para compartilhamento e comunicação eficaz.

A construção de Word Clouds é uma técnica versátil e visualmente impactante que permite explorar e comunicar padrões de palavras em conjuntos de dados de texto de maneira acessível e rápida. A escolha de parâmetros e a customização estética podem ser ajustadas de acordo com os objetivos da análise.

O QUE VOCÊ VIU NESTA AULA?

Você aprendeu a como implementar o Bag of Words e como aplicá-lo em um modelo de predição com regressão logística usando uma base de dados de avaliações de produtos, e também aprendeu a como observar e ter insights melhores da sua base a partir da visualização gráfica com o WordCloud visualizando as palavras mais recorrentes no dataset.



REFERÊNCIAS

DIAS, F. **Brazilian Portuguese Sentiment Analysis Datasets**. 2021. Disponível em: <<https://www.kaggle.com/datasets/fredericods/ptbr-sentiment-analysis-datasets?select=buscape.csv>> Acesso em: 17 jan. 2024.

SCIKIT LEARN. **Documentação Scikit Learn train_test_split**. [s.d]. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html> Acesso em: 17 jan. 2024.

SCIKIT LEARN. **Documentação Scikit Learn LogisticRegression**. [s.d]. Disponível em: <https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html> Acesso em: 17 jan. 2024.

WORDCLOUD. **Documentação WordCloud**. [s.d]. Disponível em: <https://amueller.github.io/word_cloud/>. Acesso em: 17 jan. 2024.

PALAVRAS-CHAVE

Bag of Words. WordCloud. PLN.

EMANIP



POSTECH